Stocker - A Cloud based stock trading application using Flask and AWS

1. Project Overview

Stocker is a next-generation, cloud-native stock trading platform designed to provide fast, secure, and reliable digital investing. Users can register, log in, execute trades, track portfolios, and view transaction history in real-time.

Key Features:

- Secure user registration & login
- Real-time buy & sell of stocks
- Portfolio dashboard & transaction history
- Cloud-native deployment on AWS

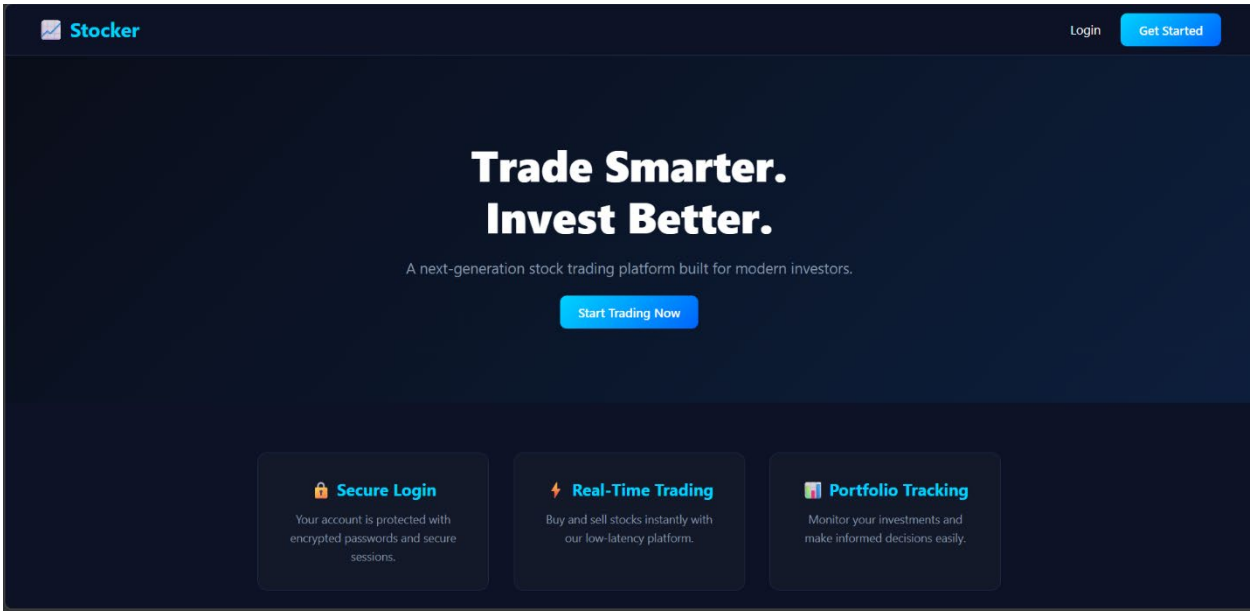
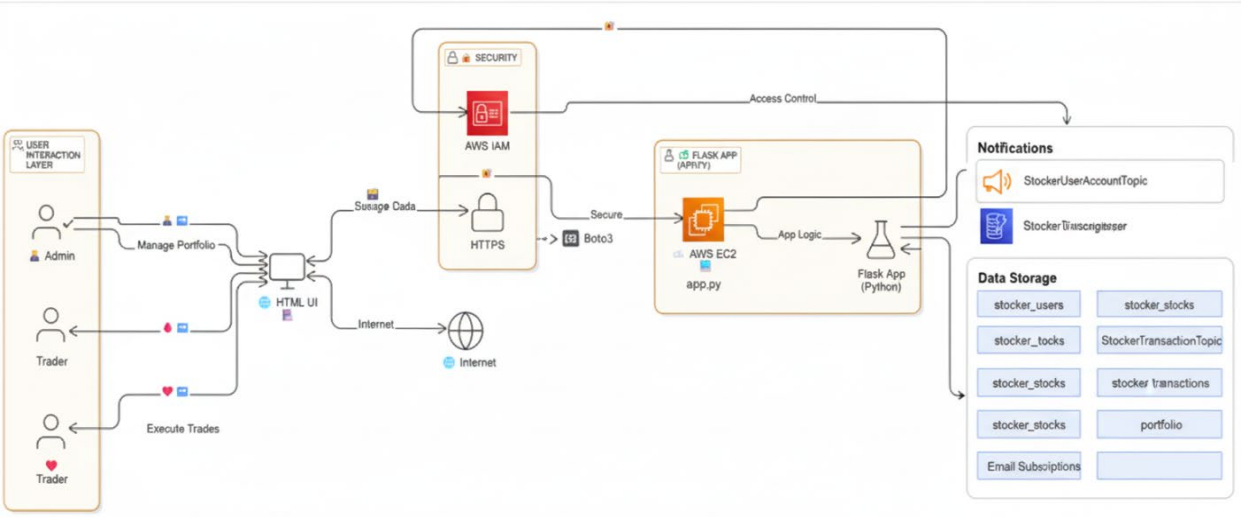


Figure 1: Stocker Home Page

2. Technical Architecture



2.1 Frontend

- Developed using Flask templates (render_template)
- HTML forms for registration, login, trading, and portfolio views
- Flash messages provide immediate feedback to users

2.2 Backend

- Flask: Handles routing, session management, and dynamic content
- bcrypt: Secures passwords using hashed storage
- UUID & datetime: Generate unique transaction IDs and timestamps
- Session Management: Maintains user state and transaction data in real-time

2.3 Database & Cloud Integration

- AWS DynamoDB (via boto3) for persistent storage:
    - User profiles
    - Trade history
    - Portfolio holdings
    - Orders
- Tables structured with keys like UserID, OrderID, AssetID for fast access
- Replaces in-memory session storage for production-ready data persistence

2.4 Cloud Deployment

- Hosted on AWS EC2 for scalable, reliable cloud access
- Configured with Gunicorn + Nginx for production-ready deployment
- Environment variables store secure AWS credentials and Flask secret key

2.5 Monitoring & Optimization

- AWS CloudWatch: Monitors system performance, latency, and API activity
- Backend optimized for low-latency order execution
- Scalable architecture handles high trading volumes

3. Environment Setup

- Install Python 3.12
- Install required libraries:

  ```
  pip install flask bcrypt boto3
  ```
- Create config.py with AWS credentials and Flask secret key
- Launch Flask app:

  ```
  python app.py
  ```

- Open in browser: http://127.0.0.1:5000

4. Core Modules & Features

4.1 User Authentication

- Secure registration with hashed passwords (bcrypt)
- Session-based login and logout
- Flash messages for feedback
- User data persisted in DynamoDB

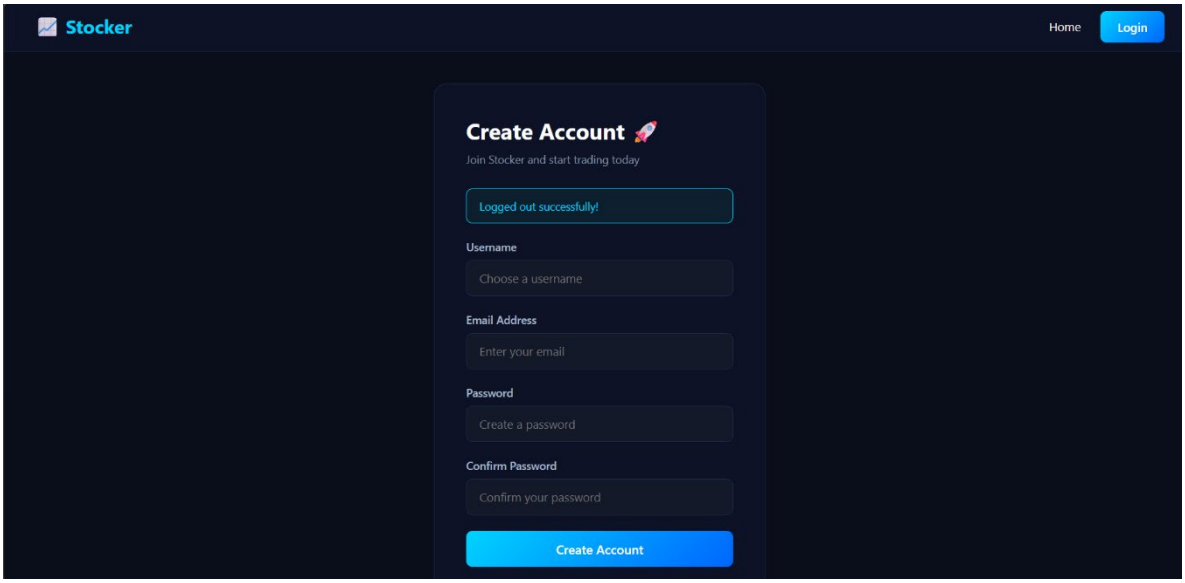


Figure 2: User Registration Page

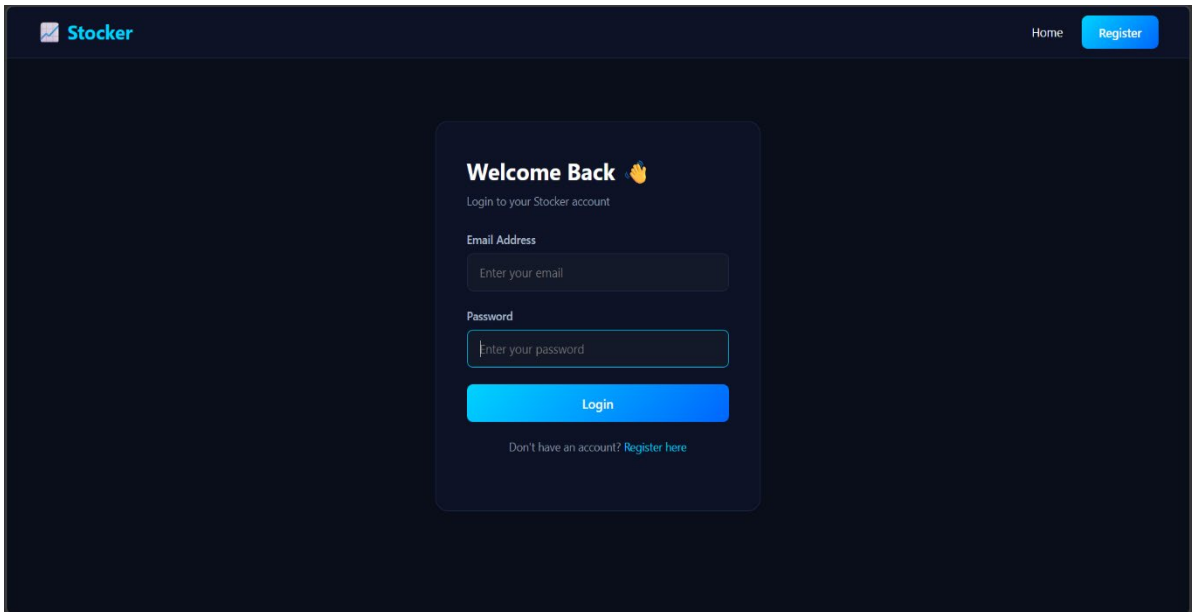


Figure 3: User Login Page

4.2 Trade Execution

- Buy and sell stocks in real-time
- Validate user balance before purchases
- Transactions recorded in DynamoDB with timestamps

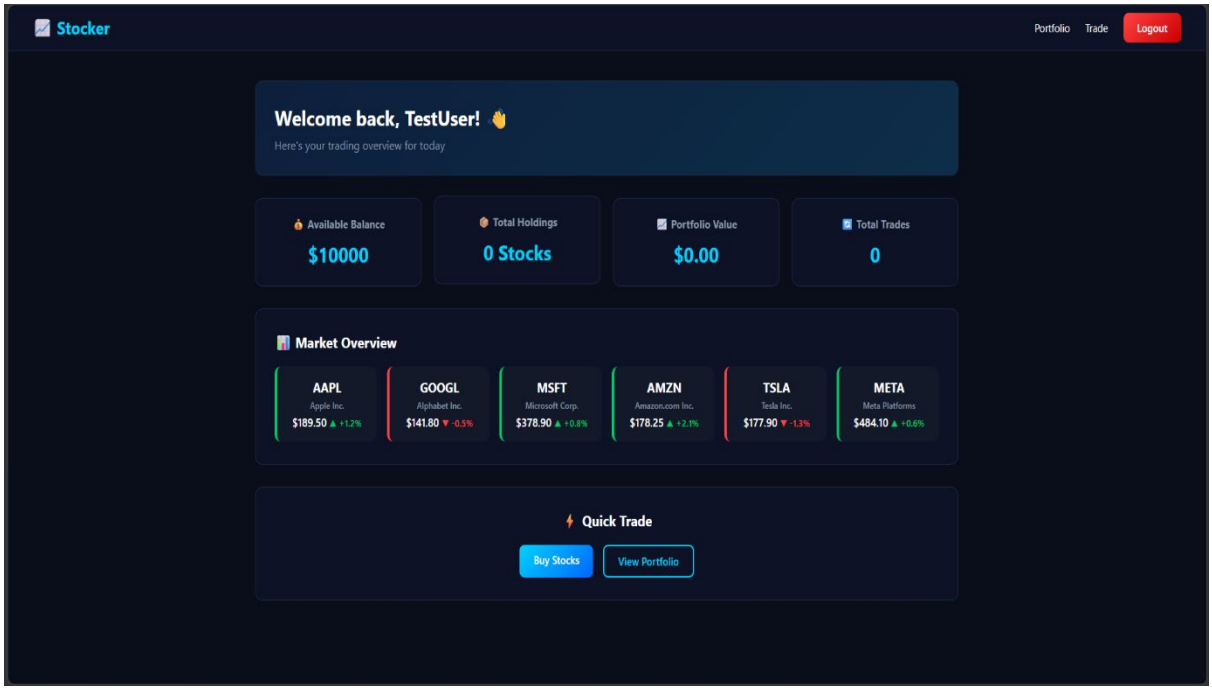


Figure 4: Stock Trading Dashboard

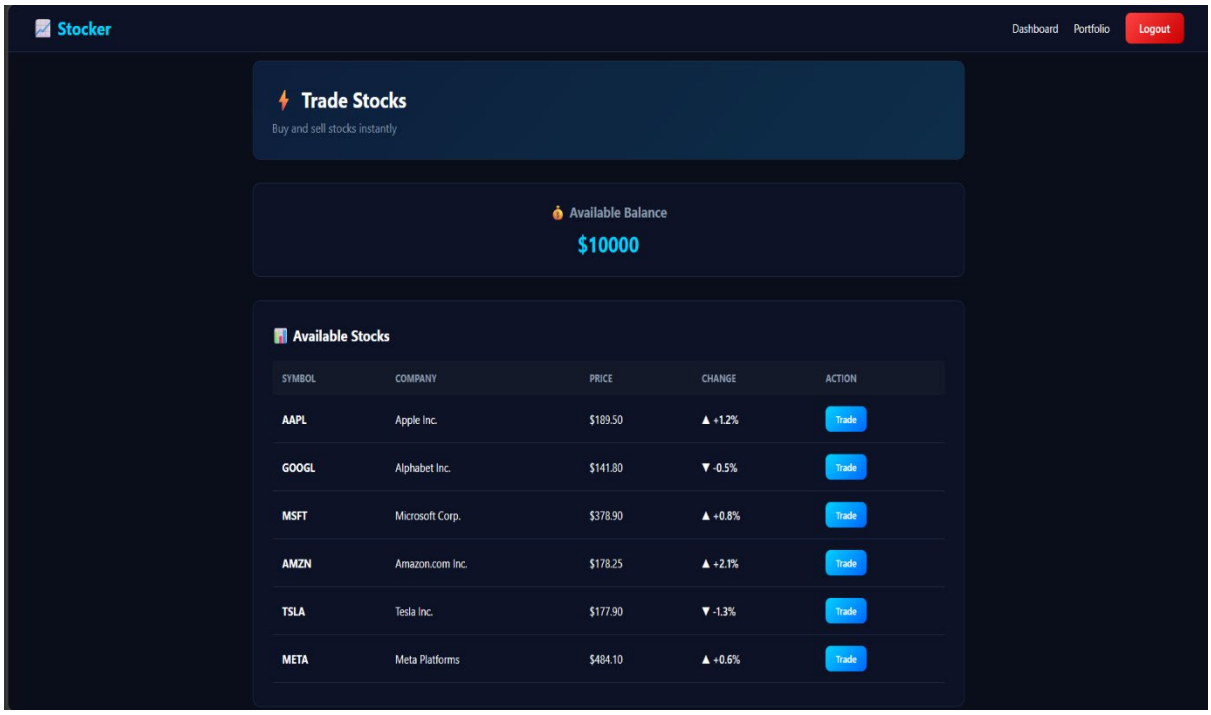


Figure 5: List of stocks available for purchase with current prices and selection options

4.3 Portfolio Management

- Dashboard displays current holdings, quantity, average price, and total value
- Calculate profit/loss for each asset
- Real-time portfolio data fetched from DynamoDB

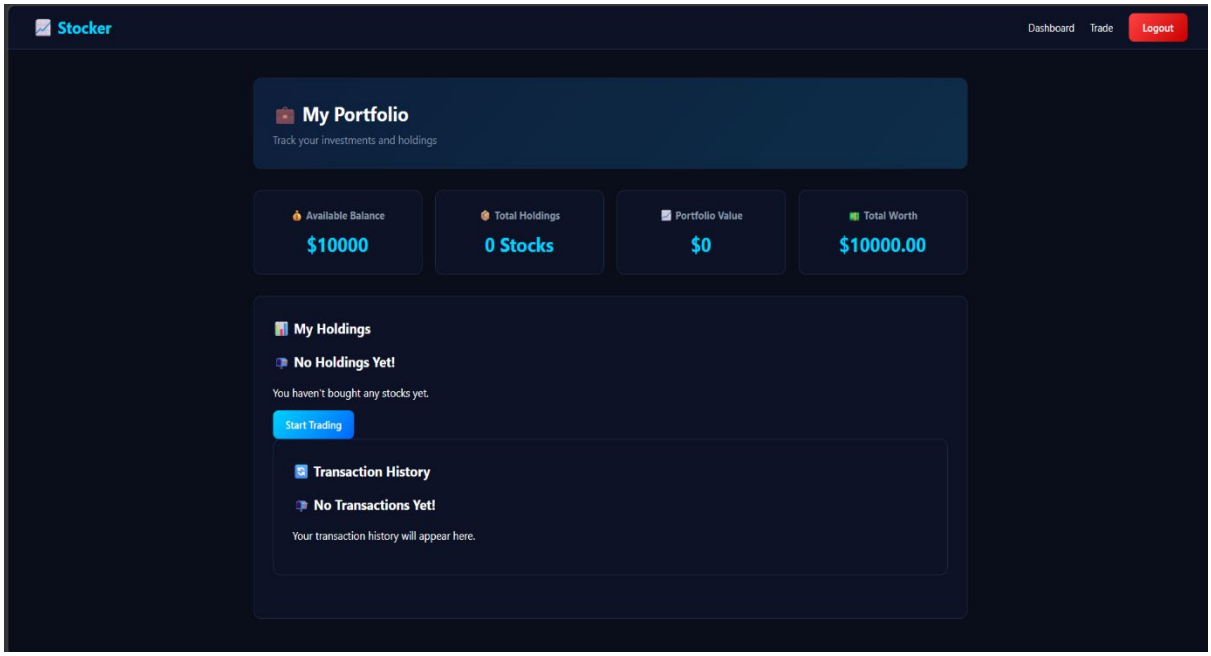


Figure 6: Portfolio Dashboard

4.4 Transaction History

- Track all buy/sell operations
- Display symbol, company name, quantity, price, total cost, and transaction date
- Stored in DynamoDB for persistence and analytics

5. AWS Integration

- boto3 connects Flask app with DynamoDB for data persistence
- EC2 used for hosting the Flask app
- CloudWatch monitors system performance and API activity
- Environment variables secure AWS credentials

6. Demo Instructions

- Run the app locally:

  python app.py
- Open browser: http://127.0.0.1:5000/
- Demonstrate:

    - User registration & login
    - Buying & selling stocks
    - Portfolio dashboard & transaction history
    - Logging out
- For cloud deployment, replace localhost URL with EC2 public IP address

7. Outcome & Learning

- Built a real-time stock trading platform using Flask and AWS
- Implemented secure authentication and session management
- Developed interactive portfolio tracking dashboards
- Integrated DynamoDB for persistent data storage
- Deployed and optimized cloud-native application using EC2 & DynamoDB
- Gained experience in full-stack development, cloud deployment, and fintech application design